

Informale Beschreibungen der Algorithmen Strukturen in der Algebra - Schwerpunkte und effiziente Methoden

Stephan Epp

22. September 2026

Inhaltsverzeichnis

1	Vektor-Schwerpunkt-Erkennung	3
1.1	Intuitive Erklärung	3
1.2	Schritt-für-Schritt-Erklärung	3
1.3	Beispiel	3
1.4	Komplexität	3
2	Matrix-Schwerpunkt-Erkennung	3
2.1	Intuitive Erklärung	3
2.2	Schritt-für-Schritt-Erklärung	4
2.3	Beispiel	4
2.4	Komplexität	4
3	Multiplikationslängen (MLM)	5
3.1	Intuitive Erklärung	5
3.2	Detaillierte Erklärung jeder Multiplikationslänge	5
3.2.1	ML-1: Direkte Multiplikation	5
3.2.2	ML-2: Schwerpunkt-Rebalancierung	5
3.2.3	ML-3: Strukturelle Rotation	5
3.2.4	ML-4: Zweite Rebalancierung	6
3.2.5	ML-5: Indirekte Effekte	6
3.3	Schritt-für-Schritt-Algorithmus	6
3.4	Komplexität	6
4	Optimale Rotation zur Blockstruktur-Minimierung	6
4.1	Intuitive Erklärung	6
4.2	Schritt-für-Schritt-Erklärung	7
4.3	Was ist eine Givens-Rotation?	7
4.4	Komplexität	7

5	Schwerpunkt-Diagonalisierung	7
5.1	Intuitive Erklärung	7
5.2	Schritt-für-Schritt-Erklärung	8
5.3	Warum funktioniert das?	8
5.4	Komplexität	8
6	Iterative Schwerpunkt-Verfeinerung	8
6.1	Intuitive Erklärung	8
6.2	Schritt-für-Schritt-Erklärung	8
6.3	Konvergenz	9
6.4	Komplexität	9
7	QR-Algorithmus mit Schwerpunkt-Vorkonditionierung	9
7.1	Intuitive Erklärung	9
7.2	Schritt-für-Schritt-Erklärung	9
7.3	Konvergenz	9
7.4	Komplexität	10
8	Schwerpunkt-Vorkonditionierung für Lineare Gleichungssysteme	10
8.1	Intuitive Erklärung	10
8.2	Schritt-für-Schritt-Erklärung	10
8.3	Effekt	10
8.4	Komplexität	10
9	Strukturelle Unsicherheit	11
9.1	Intuitive Erklärung	11
9.2	Entropie-Berechnung	11
9.3	Spektrallücken-Berechnung	11
9.4	Gesamt-Unsicherheit	11
10	Zusammenfassung	12

1 Vektor-Schwerpunkt-Erkennung

1.1 Intuitive Erklärung

Der Schwerpunkt eines Vektors ist ganz einfach das Element, das den größten Absolutwert hat. Stell dir einen Vektor vor wie eine Liste von Zahlen: $(1.5, 0.3, 7.2, -2.1, 0.8)$. Der Schwerpunkt ist hier die Zahl 7.2, weil sie betragsmäßig am größten ist.

Warum ist das sinnvoll? Das Element mit der maximalen Magnitude repräsentiert die "dominanteste" Komponente des Vektors. In vielen Anwendungen ist dies der beste Kandidat, um die Struktur des Vektors zu verstehen oder zu charakterisieren.

1.2 Schritt-für-Schritt-Erklärung

1. **Initialisierung:** Starte mit $\text{max_value} = 0$ und $\text{max_index} = 0$.
2. **Durchlaufen:** Gehe durch jeden Eintrag des Vektors. Wenn der Absolutwert des aktuellen Elements größer ist als das bisherige Maximum, merke dir diesen Index und diesen Wert.
3. **Rückgabe:** Gib Index und Wert des Maximums zurück.

1.3 Beispiel

Für den Vektor $v = (0.1, 0.2, 5.3, 0.3, 0.5)$ läuft der Algorithmus wie folgt:

- Element 0: $|0.1| = 0.1 \rightarrow$ Update max
- Element 1: $|0.2| = 0.2 \rightarrow$ Update max
- Element 2: $|5.3| = 5.3 \rightarrow$ Update max (dies ist das Maximum!)
- Element 3: $|0.3| = 0.3 \rightarrow$ Nicht größer
- Element 4: $|0.5| = 0.5 \rightarrow$ Nicht größer

Ergebnis: Index = 2, Wert = 5.3

1.4 Komplexität

- **Zeitkomplexität:** $O(n)$ — man muss jeden der n Einträge genau einmal anschauen
- **Speicherkomplexität:** $O(1)$ — man speichert nur zwei Zahlen (Index und Wert)

2 Matrix-Schwerpunkt-Erkennung

2.1 Intuitive Erklärung

Der Schwerpunkt einer Matrix ist ihre Diagonale — also die Elemente von links oben nach rechts unten. Zusätzlich berechnet man einen Index namens DSI (Diagonalisierungs-Schwerpunkt-Index), der misst, wie konzentriert die "Energie" der Matrix auf ihrer Diagonale liegt.

Warum die Diagonale? Die Diagonale enthält die Selbstinteraktionen jeder Komponente. Sie bestimmt fundamentale Eigenschaften wie die Spur (Summe der Diagonale) oder den Großteil der Eigenwerte (wenn die Matrix diagonalisierbar ist).

2.2 Schritt-für-Schritt-Erklärung

1. **Diagonale auslesen:** Laufe durch alle Indizes i von 1 bis n und speichere A_{ii} (das Element an Position (i, i)).
2. **Diagonalenergie berechnen:** Summiere die Quadrate aller Diagonalelemente: $\sum_i A_{ii}^2$.
3. **Gesamtenergie berechnen:** Summiere die Quadrate aller Matrixelemente (das ist die Frobenius-Norm zum Quadrat): $\sum_{i,j} A_{ij}^2$.
4. **DSI berechnen:** Teile die Diagonalenergie durch die Gesamtenergie:
$$\text{DSI} = \frac{\text{Diagonalenergie}}{\text{Gesamtenergie}}.$$
5. **Rückgabe:** Gib den Diagonalvektor und den DSI zurück.

2.3 Beispiel

Für die Matrix

$$A = \begin{pmatrix} 4 & 0.1 & 0.2 \\ 0.1 & 3 & 0.1 \\ 0.2 & 0.1 & 5 \end{pmatrix}$$

läuft der Algorithmus wie folgt:

- Diagonale: $(4, 3, 5)$
- Diagonalenergie: $4^2 + 3^2 + 5^2 = 16 + 9 + 25 = 50$
- Gesamtenergie: $16 + 0.01 + 0.04 + 0.01 + 9 + 0.01 + 0.04 + 0.01 + 25 = 50.12$
- DSI: $50/50.12 \approx 0.998$

Ein DSI von 0.998 bedeutet, dass fast 100% der Energie auf der Diagonale liegt — die Matrix ist praktisch bereits diagonal!

2.4 Komplexität

- **Zeitkomplexität:** $O(n^2)$ — man muss alle $n \times n$ Elemente durchlaufen
- **Speicherkomplexität:** $O(n)$ — man speichert den Diagonalvektor

3 Multiplikationslängen (MLM)

3.1 Intuitive Erklärung

Die Multiplikationslängen sind eine Sequenz von fünf gestaffelten Transformationen zweier Grundmatrizen A und B . Jede Transformation enthüllt tiefere strukturelle Informationen. Das Ziel ist, das System durch Nutzung von Schwerpunkten (Diagonale) zu vereinfachen und zu stabilisieren.

Die fünf Längen:

1. **ML-1:** Die direkte Multiplikation $M_1 = A \times B$
2. **ML-2:** Normalisierung durch Zeilen- und Spaltensummen (Schwerpunkt-Rebalancierung)
3. **ML-3:** Rotation der Struktur (optimale Reorganisation der Sektoren)
4. **ML-4:** Zweite Normalisierung nach Rotation
5. **ML-5:** Selbstmultiplikation $M_5 = M_4 \times M_4$ (indirekte Effekte)

3.2 Detaillierte Erklärung jeder Multiplikationslänge

3.2.1 ML-1: Direkte Multiplikation

Das ist einfach die Standard-Matrixmultiplikation:

$$M_1 = A \times B$$

Der Schwerpunkt von ML-1 ist die Diagonale $\text{diag}(M_1)$.

3.2.2 ML-2: Schwerpunkt-Rebalancierung

Normalisiere die Matrix nach ihren Zeilen- und Spaltensummen:

$$M_2 = D_L^{-1} \times M_1 \times D_R^{-1}$$

wobei D_L die Diagonale der Zeilensummen und D_R die Diagonale der Spaltensummen sind.

Intuition: Wenn eine Zeile viel größer ist als die anderen, wird sie runterskaliert; ebenso für Spalten. Das macht alle Sektoren vergleichbar.

Effekt: Nach dieser Transformation sollten die Zeilensummen ungefähr 1 sein.

3.2.3 ML-3: Strukturelle Rotation

Wende eine optimale orthogonale Rotation R auf M_2 an:

$$M_3 = R^T \times M_2 \times R$$

Die Rotation wird so gewählt, dass die Off-Diagonalelemente minimiert werden.

Intuition: Das ist wie das Neuordnen der Sektoren, um versteckte Cluster oder hierarchische Strukturen sichtbar zu machen.

3.2.4 ML-4: Zweite Rebalancierung

Wende die gleiche Rebalancierung wie bei ML-2 an, aber jetzt auf M_3 :

$$M_4 = D_L^{-1} \times M_3 \times D_R^{-1}$$

Effekt: Nach der Rotation sind die Sektor-Größen verändert, also normalisiert man erneut.

3.2.5 ML-5: Indirekte Effekte

Quadriere die Matrix:

$$M_5 = M_4 \times M_4$$

Intuition: Das zeigt, was passiert, wenn das System mit sich selbst "wechselwirkt" über zwei Schritte iteriert wird. Kleine Eigenwerte werden noch kleiner, große werden größer.

3.3 Schritt-für-Schritt-Algorithmus

1. Berechne $M_1 = A \times B$
2. Berechne Zeilensummen $\ell_i = \sum_j M_1[i, j]$ und Spaltensummen $r_j = \sum_i M_1[i, j]$
3. Erstelle Diagonalmatrizen $D_L = \text{diag}(\ell)$ und $D_R = \text{diag}(r)$
4. Berechne $M_2 = D_L^{-1} \times M_1 \times D_R^{-1}$
5. Finde optimale Rotationsmatrix R (siehe Algorithmus ??)
6. Berechne $M_3 = R^T \times M_2 \times R$
7. Wiederhole Schritte 2-4 für M_3 um M_4 zu erhalten
8. Berechne $M_5 = M_4 \times M_4$

3.4 Komplexität

- **ML-1, ML-2, ML-4:** $O(n^3)$ für Matrixmultiplikationen
- **ML-3:** $O(n^4)$ (oder $O(n^3)$ mit vereinfachter Rotation)
- **ML-5:** $O(n^3)$

4 Optimale Rotation zur Blockstruktur-Minimierung

4.1 Intuitive Erklärung

Dieser Algorithmus findet die beste Rotationsmatrix R , um eine Matrix so zu drehen, dass ihre Off-Diagonalelemente minimiert werden. Mit anderen Worten: Wir suchen eine Rotation, die die Matrix "näher zur Diagonale" bringt.

Warum rotieren? Manche Strukturen sind versteckt. Eine Matrix kann in ihrer ursprünglichen Darstellung vollständig dicht sein, aber wenn man sie klug rotiert, entstehen plötzlich Blöcke auf der Diagonale.

4.2 Schritt-für-Schritt-Erklärung

1. **Initialisierung:** Starte mit $R = I$ (Identitätsmatrix)
2. **Iterationen:** Wiederhole bis Konvergenz (max. Iterationen):
 - (a) Für alle Paare (i, j) mit $i < j$:
 - i. Berechne den optimalen Winkel θ für eine Givens-Rotation in der (i, j) -Ebene
 - ii. Wende diese Rotation an: $R \leftarrow R \times \text{Givens}(i, j, \theta)$
 - (b) Berechne die Norm der Off-Diagonalelemente (Frobenius-Norm)
 - (c) Falls diese sich weniger als ein Schwellwert geändert hat, stop (konvergiert)

4.3 Was ist eine Givens-Rotation?

Eine Givens-Rotation rotiert nur in einer 2D-Ebene. Sie hat die Form:

$$G(i, j, \theta) = \begin{pmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots \\ \vdots & \ddots & \vdots & & \vdots & \\ 0 & \cdots & \cos \theta & \cdots & -\sin \theta & \cdots \\ \vdots & & \vdots & \ddots & \vdots & \\ 0 & \cdots & \sin \theta & \cdots & \cos \theta & \cdots \\ \vdots & & \vdots & & \vdots & \ddots \end{pmatrix}$$

Intuition: Sie mischt nur zwei Komponenten (bei Positionen i und j) und lässt alles andere unverändert.

4.4 Komplexität

- **Zeitkomplexität:** $O(n^3 \cdot k)$, wobei k die Zahl der Iterationen ist (typisch klein)
- **Speicherkomplexität:** $O(n^2)$ für die Rotationsmatrix

5 Schwerpunkt-Diagonalisierung

5.1 Intuitive Erklärung

Der Zweck dieses Algorithmus ist, eine Matrix in eine (nahezu) diagonale Form zu transformieren. Das wird iterativ gemacht: In jedem Schritt wird die Singulärwertzerlegung (SVD) durchgeführt und die Matrix wird transformiert, bis der Diagonalisierungs-Schwerpunkt-Index (DSI) groß ist.

Warum Diagonalisierung? Viele algebraische Operationen sind trivial, wenn die Matrix diagonal ist. Daher ist Diagonalisierung ein universelles Werkzeug.

5.2 Schritt-für-Schritt-Erklärung

1. **Initialisierung:** Setze $Q = I$ und $B = A$
2. **Hauptschleife:** Wiederhole für $k = 1, 2, \dots, \text{max_iterations}$:
 - (a) Berechne den DSI von B (siehe Algorithmus 2)
 - (b) Falls $\text{DSI} > 0.95$, dann ist die Matrix quasi-diagonal $\rightarrow$ Stopp und gib (B, Q) zurück
 - (c) Berechne die Singulärwertzerlegung: $B = U\Sigma V^T$
 - (d) Transformiere: $B \leftarrow U^T B V$
 - (e) Akkumuliere: $Q \leftarrow Q V$
3. **Rückgabe:** Die diagonalisierte (oder quasi-diagonalisierte) Matrix B und die Transformationsmatrix Q

5.3 Warum funktioniert das?

Die SVD zerlegt die Matrix in ihre Struktur. Das Produkt $U^T B V$ "rotiert" die Matrix optimiert. Mit jeder Iteration konzentriert sich die Energie mehr auf der Diagonale.

5.4 Komplexität

- **Zeitkomplexität:** $O(k \cdot n^3)$, wobei k typisch klein ist (3-10 Iterationen)
- **Speicherkomplexität:** $O(n^2)$

6 Iterative Schwerpunkt-Verfeinerung

6.1 Intuitive Erklärung

Dieser Algorithmus macht schrittweise Fortschritt, indem er in jeder Iteration die Matrix durch eine Mischung aus sich selbst und ihrer Diagonale ersetzt. Das ist viel schneller als die SVD-basierte Diagonalisierung, aber funktioniert nur, wenn $\|A\|_2 \leq 1$.

Intuition: Wenn du zu 50% die Diagonale behältst und zu 50% die ganze Matrix, werden die Off-Diagonalelemente Schritt für Schritt kleiner.

6.2 Schritt-für-Schritt-Erklärung

1. **Parameter:** Wähle einen Mischungsparameter $\alpha \in (0, 1)$, z.B. $\alpha = 0.5$
2. **Initialisierung:** Setze $A^{(0)} = A$
3. **Hauptschleife:** Wiederhole für $k = 0, 1, 2, \dots, \text{max_iterations}$:
 - (a) Extrahiere die Diagonale: $D^{(k)} = \text{diag}(A^{(k)})$
 - (b) Mische: $A^{(k+1)} = \alpha \cdot A^{(k)} + (1 - \alpha) \cdot D^{(k)}$
 - (c) Berechne den Fehler: $\text{error} = \|A^{(k+1)} - D^{(k)}\|_F / \|D^{(k)}\|_F$

- (d) Falls $\text{error} < \text{tolerance}$, stopp
- (e) Setze $A^{(k)} \leftarrow A^{(k+1)}$

4. **Rückgabe:** Die verfeinerte Matrix $A^{(\infty)}$ und die Fehler-Historie

6.3 Konvergenz

Der Fehler nimmt exponentiell ab mit Rate α . Mit $\alpha = 0.5$ halbiert sich der Fehler ungefähr in jeder Iteration. Man braucht also etwa $\log_2(\epsilon^{-1})$ Iterationen für Genauigkeit ϵ .

6.4 Komplexität

- **Zeitkomplexität:** $O(k \cdot n^2)$, wobei $k \approx 3 \log_{10}(\epsilon^{-1})$ ist
- **Speicherkomplexität:** $O(n^2)$

7 QR-Algorithmus mit Schwerpunkt-Vorkonditionierung

7.1 Intuitive Erklärung

Der QR-Algorithmus ist das Standard-Verfahren zur Eigenwertberechnung. Die Idee: Wiederhole $A \rightarrow QR$ (QR-Zerlegung) und dann $A \rightarrow RQ$. Mit jeder Iteration wird A näher zu einer Diagonalmatrix mit den Eigenwerten auf der Diagonale.

Verbesserung durch Schwerpunkt: Vor der QR-Schleife vorkonditionieren wir die Matrix durch Skalierung mit ihrer Diagonale. Das beschleunigt die Konvergenz erheblich.

7.2 Schritt-für-Schritt-Erklärung

1. **Schwerpunkt extrahieren:** Extrahiere die Diagonale $M = \text{diag}(A)$
2. **Vorkonditionierung:** Skaliere $A' \leftarrow M^{-1}AM$
3. **QR-Schleife:** Wiederhole für $k = 1, 2, \dots, \text{max_iter}$:
 - (a) Berechne die QR-Zerlegung: $A' = QR$
 - (b) Aktualisiere: $A' \leftarrow RQ$
 - (c) Akkumuliere die Transformation: $Q_{\text{total}} \leftarrow Q_{\text{total}}Q$
 - (d) Konvergenz-Check: Falls $\|A' - \text{diag}(A')\|_F < \text{tol}$, stopp
4. **Eigenwerte:** Die Diagonale von A' sind die Eigenwerte
5. **Eigenvektoren:** Berechne $P = M \cdot Q_{\text{total}}$

7.3 Konvergenz

Ohne Vorkonditionierung konvergiert der QR-Algorithmus mit Rate $|\lambda_2/\lambda_1|$. Mit Schwerpunkt-Vorkonditionierung wird diese Rate typischerweise um Faktor 2–3 verbessert.

7.4 Komplexität

- **Zeitkomplexität:** $O(k \cdot n^3)$, wobei k die Zahl der QR-Iterationen ist
- **Speicherkomplexität:** $O(n^2)$

8 Schwerpunkt-Vorkonditionierung für Lineare Gleichungssysteme

8.1 Intuitive Erklärung

Um das System $Ax = b$ zu lösen, kann man es vorkonditionieren, indem man es transformiert zu $M^{-1}Ax = M^{-1}b$, wobei M eine einfache Matrix ist. Wenn wir $M = \text{diag}(A)$ wählen (die Diagonale), ist M^{-1} trivial zu berechnen und verbessert die Kondition des Problems.

Intuition: Wenn die Matrix sehr unausgeglich ist (manche Zeilen viel größer als andere), skalieren wir sie, um sie ausgeglichener zu machen. Das macht iterative Löser viel schneller.

8.2 Schritt-für-Schritt-Erklärung

1. **Vorkonditioner extrahieren:** Setze $M = \text{diag}(A)$, und ersetze Nullen durch 1
2. **Inverse berechnen:** $M^{-1} = \text{diag}(1/m_1, 1/m_2, \dots, 1/m_n)$
3. **System transformieren:**

$$A' \leftarrow M^{-1}A \tag{1}$$

$$b' \leftarrow M^{-1}b \tag{2}$$

4. **System lösen:** Löse $A'x = b'$ mit beliebiger Methode (direkt, iterativ, etc.)
5. **Rückgabe:** Die Lösung x

8.3 Effekt

Die Vorkonditionierung reduziert die Konditionszahl $\kappa(A)$ typischerweise um einen Faktor 2–10. Dies macht die Lösung schneller und numerisch stabiler.

8.4 Komplexität

- **Vorkonditioner-Setup:** $O(n)$
- **Lösungszeit:** Abhängig von der Lösungsmethode, typisch $O(k \cdot n^2)$ für iterative Methoden

9 Strukturelle Unsicherheit

9.1 Intuitive Erklärung

Je komplizierter eine Struktur (wie eine Multiplikationslänge), desto mehr Unsicherheit ist mit ihr verbunden. Die strukturelle Unsicherheit besteht aus drei Komponenten:

1. **Entropie:** Wie "verteilt" sind die Einträge? Wenn alles auf der Diagonale liegt, ist die Entropie niedrig. Wenn alles diffus ist, ist sie hoch.
2. **Spektrallücke:** Wie schnell konvergiert das System? Eine große Spektrallücke bedeutet schnelle Konvergenz.
3. **Sensitivität:** Wie sehr ändert sich die Matrix, wenn wir die Eingaben leicht ändern?

9.2 Entropie-Berechnung

1. **Zeilennormalisierung:** Normalisiere jede Zeile so, dass die Zeilensumme 1 ist
2. **Shannon-Entropie:** Für jedes normalisierte Element p_{ij} , berechne:

$$H = - \sum_{i,j} p_{ij} \log(p_{ij})$$

3. Die Entropie H ist zwischen 0 (stark strukturiert) und $\log(n^2)$ (völlig chaotisch)

9.3 Spektrallücken-Berechnung

1. Berechne die Eigenwerte der Matrix
2. Sortiere sie nach Absolutwert (absteigend)
3. Die Spektrallücke ist: $\text{gap} = 1 - |\lambda_2|$
4. Eine große Lücke bedeutet gute Konvergenz

9.4 Gesamt-Unsicherheit

Addiere alle drei Komponenten:

$$U_k = \mathcal{H}(M_k) + \mathcal{S}(M_k) + \mathcal{G}(M_k)$$

Mit wachsender Multiplikationslänge k nimmt U_k exponentiell zu. Das ist das PParadox der Multiplikationslänge.

10 Zusammenfassung

Die nachfolgende Beschreibung erläutert die in Tabelle 1 zusammengefassten neun Verfahren der numerischen und strukturellen Algebra. Die Verfahren sind nach ihren Eingabedaten, primären Einsatzzwecken sowie ihrer Laufzeit- und Speicherkomplexität strukturiert. Die Komplexitätsangaben sind von grundlegender Bedeutung für die praktische Anwendbarkeit: Während Vektor- und Matrix-Operationen mit polylogarithmischer oder polynomialer Laufzeit das Rückgrat effizienter Algorithmen bilden, erfordern Verfahren mit kubischer oder höherer Komplexität (wie die SVD-basierte Centroid-Diagonalisierung) typischerweise spezialisierte Vorkonditionierungen oder iterative Beschleunigungstechniken.

1. Vektor-Schwerpunkt (Eingabe: Vektor)

Verwendung: Finde dominanteste Komponente

Komplexität: Zeit $O(n)$, Speicher $O(1)$

Durchläuft ein-dimensionale Datenstrukturen und ermittelt das Element mit der größten absoluten Magnitude. Dies dient dazu, die bedeutendste Komponente oder den Hauptträger der „Energie“ im Vektor effizient zu identifizieren. Mit linearer Laufzeit und konstantem Speicheroverhead ist dieses Verfahren hochgradig skalierbar.

2. Matrix-Schwerpunkt (Eingabe: Matrix)

Verwendung: Messe wie diagonal die Matrix ist

Komplexität: Zeit $O(n^2)$, Speicher $O(n)$

Bewertet zweidimensionale Datenstrukturen, indem die Hauptdiagonale isoliert und ins Verhältnis zur Gesamtmatrix gesetzt wird. Das Ergebnis (z. B. der Diagonalisierungs-Schwerpunkt-Index, DSI) beschreibt quantitativ, wie stark die Matrix bereits um ihre Diagonale zentriert ist. Die quadratische Laufzeit ist erforderlich, um alle Matrixelemente zu durchlaufen.

3. Multiplikationslängen (Eingabe: 2 Matrizen)

Verwendung: Analysiere strukturelle Verflechtungen

Komplexität: Zeit $O(n^3)$, Speicher $O(n^2)$

Nimmt zwei Matrizen entgegen und durchläuft eine mehrstufige Transformationssequenz (u. a. Verknüpfung, Rebalancierung über Zeilen-/Spaltensummen und orthogonale Rotation). Dadurch werden verdeckte Wechselwirkungen, Cluster und Pfadabhängigkeiten zwischen den Eingabesystemen aufgedeckt. Die kubische Komplexität resultiert aus der Matrixmultiplikation in jedem Transformationsschritt.

4. Optimal-Rotation (Eingabe: Matrix)

Verwendung: Transformiere Matrix blockweise diagonal

Komplexität: Zeit $O(k \cdot n^2)$, Speicher $O(n^2)$

Wendet schrittweise ebene Drehungen (Givens-Rotationen) auf eine Matrix an, um die Werte außerhalb der Diagonale gezielt zu minimieren. Dies bringt versteckte Blockstrukturen ans Licht und bereitet die Matrix für weitere Zerlegungen vor. Die Laufzeit hängt von der Anzahl k der Iterationen ab, die zur Konvergenz erforderlich sind.

5. Centroid-Diagonalisierung (Eingabe: Matrix)

Verwendung: Diagonalisiere mit SVD iterativ

Komplexität: Zeit $O(k \cdot n^3)$, Speicher $O(n^2)$

Ein iteratives Verfahren, das in jedem Schritt eine Singulärwertzerlegung (SVD)

Algorithmus	Eingabe	Verwendung	Laufzeit	Speicher
Vektor-Schwerpunkt	Vektor	Finde dominanteste Komponente	$O(n)$	$O(1)$
Matrix-Schwerpunkt	Matrix	Messe wie diagonal die Matrix ist	$O(n^2)$	$O(n)$
Multiplikationslängen	2 Matrizen	Analysiere strukturelle Verflechtungen	$O(n^3)$	$O(n^2)$
Optimal-Rotation	Matrix	Transformiere Matrix blockweise diagonal	$O(k \cdot n^2)$	$O(n^2)$
Centroid-Diagonalisierung	Matrix	Diagonalisiere mit SVD iterativ	$O(k \cdot n^3)$	$O(n^2)$
Schwerpunkt-Verfeinerung	Matrix	Verfeinere zu Diagonale (schnell)	$O(k \cdot n^2)$	$O(n^2)$
QR-Centroid	Matrix	Berechne Eigenwerte/vektoren	$O(k \cdot n^3)$	$O(n^2)$
Vorkonditionierung	Gleichungssystem	Beschleunige Lösungsverfahren	$O(n) + O(k \cdot n^2)$	$O(n)$
Strukturelle Unsicherheit	Matrizen	Messe Zuverlässigkeit von Vorhersagen	$O(n^2)$	$O(n^2)$

Tabelle 1: Übersicht der Algorithmen mit Komplexitätsanalyse

nutzt, um die Matrix stufenweise in eine quasi-diagonale Gestalt zu überführen. Es wird beendet, sobald der Diagonalisierungsgrad einen definierten Schwellenwert erreicht. Die kubische Komplexität pro Iteration wird durch die SVD-Berechnung verursacht.

6. Schwerpunkt-Verfeinerung (Eingabe: Matrix)

Verwendung: Verfeinere zu Diagonale (schnell)

Komplexität: Zeit $O(k \cdot n^2)$, Speicher $O(n^2)$

Bietet für normierte Matrizen eine recheneffiziente Alternative zur SVD-basierten Diagonalisierung. Durch schrittweises Mischen der Matrix mit ihrer eigenen Diagonale wird der Off-Diagonal-Anteil exponentiell reduziert, was zu einer raschen Glättung führt. Die quadratische Laufzeit pro Iteration macht dieses Verfahren deutlich schneller als die SVD-Variante.

7. QR-Centroid (Eingabe: Matrix)

Verwendung: Berechne Eigenwerte/vektoren

Komplexität: Zeit $O(k \cdot n^3)$, Speicher $O(n^2)$

Eine Variante des klassischen QR-Algorithmus zur Spektralanalyse. Die Matrix wird vor den eigentlichen QR-Iterationen über ihre Diagonale vorkonditioniert, was die Konvergenzgeschwindigkeit bei der Bestimmung von Eigenwerten und Eigenvektoren deutlich erhöht. Trotz der kubischen Komplexität pro Iteration führt die Vorkonditionierung typischerweise zu einer Reduktion der erforderlichen Iterationen um Faktor 2–3.

8. Vorkonditionierung (Eingabe: Gleichungssystem)

Verwendung: Beschleunige Lösungsverfahren

Komplexität: Setup $O(n)$, Lösung $O(k \cdot n^2)$, Speicher $O(n)$

Setzt bei linearen Gleichungssystemen ($Ax = b$) an. Durch Skalierung mit der Inversen der Hauptdiagonale werden Größenunterschiede zwischen den Zeilen ausgeglichen, die Kondition der Systemmatrix verbessert und iterative Löser numerisch stabilisiert. Der Speicheroverhead ist minimal, da nur der Vorkonditioner selbst gespeichert werden muss.

9. Strukturelle Unsicherheit (Eingabe: Matrizen)

Verwendung: Messe Zuverlässigkeit von Vorhersagen

Komplexität: Zeit $O(n^2)$, Speicher $O(n^2)$

Untersucht komplexe Matrixstrukturen oder mehrstufige Transformationen auf ihre mathematische Stabilität. Unter Einbeziehung von Entropie, Spektrallücke und Sensitivität wird quantifiziert, wie stark Fehler oder kleine Störungen die Vorhersagegenauigkeit des Systems beeinflussen. Die quadratische Komplexität ermöglicht auch für größere Systeme eine effiziente Stabilitätsbewertung.